

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая/Лабораторная работа

Выполнил:

Свинин Артем

Группа К3342

Проверил:

Добряков Д. И.

Цель работы

Реализовать межсервисное взаимодействие посредством очередей сообщений и подключить брокер сообщений для обмена событиями между сервисами.

Что реализовано

- В проекте подключена и настроена Kafka.
- Межсервисное взаимодействие реализовано через публикацию и потребление событий.
- Сервисы обмениваются событиями о бронированиях и платежах.
- Для HTTP-взаимодействия между сервисами также используются сервисные клиенты и служебный токен X-Service-Token.

События Kafka

В системе используются следующие события:

- booking.created
- booking.completed
- booking.cancelled
- payment.success
- payment.failed

Кто публикует и кто читает

Событие	Публикует	Читает
booking.created	Booking	Payment, Communication
booking.completed	Booking	Review
booking.cancelled	Booking	Payment
payment.success	Payment	Booking, Communication

Событие	Публикует	Читает
payment.failed	Payment	Booking

Как это работает

- При создании бронирования сервис Booking публикует событие booking.created.
- Сервис Payment реагирует на это событие и обрабатывает оплату.
- При успешной оплате публикуется payment.success.
- Сервис Booking обновляет статус бронирования на подтверждённый.
- Сервис Communication создаёт или обновляет чаты на основе событий бронирования и оплаты.
- Сервис Review использует события booking.completed для проверки возможности оставить отзыв.

Kafka выбрана как брокер сообщений для асинхронного обмена данными между сервисами. Это позволяет:

- ослабить связность между сервисами;
- обрабатывать события независимо от времени ответа другого сервиса;
- масштабировать потребителей отдельно от производителей;
- хранить и повторно читать события при необходимости.

Инфраструктура

В Docker Compose поднимаются:

- zookeeper — координатор кластера Kafka
- kafka — брокер сообщений
- kafka-ui — веб-интерфейс для наблюдения за топиками и сообщениями (доступен на порту 9080)

Kafka используется вместе с набором микросервисов:

- booking-service
- payment-service
- review-service

- communication-service

ЛР3 — Контейнеризация приложения средствами Docker

Цель работы

Контейнеризовать микросервисное приложение на Go с помощью Docker и Docker Compose, а также настроить сетевое взаимодействие между сервисами.

Что реализовано

- Для проекта подготовлен набор Dockerfile для сервисов в каталоге `dockerfiles/`.
- В репозитории есть общий `docker-compose.yml`, который поднимает все сервисы, базы данных, Kafka и API Gateway.
- Каждый сервис работает в общей Docker-сети и обращается к соседним сервисам по DNS-именам контейнеров.
- Для локального доступа используется Nginx как API Gateway.

Dockerfile

Все сервисы собираются из единого многоэтапного Dockerfile. Целевой сервис передаётся через build-аргумент `SERVICE`.

```
FROM golang:1.26-alpine AS builder
```

```
ARG SERVICE
```

```
WORKDIR /app
```

```
COPY go.work go.work.sum ./
```

```
COPY . .
```

```
WORKDIR /app/services/${SERVICE}
```

```
RUN go build -o /bin/service ./cmd
```

```
FROM alpine:3.19
```

RUN apk --no-cache add ca-certificates

COPY --from=builder /bin/service /service

ENTRYPOINT ["/service"]

На первом этапе (**builder**) компилируется бинарник нужного сервиса. На втором — собирается минимальный образ на базе **alpine:3.19**, в который копируется только скомпилированный бинарник. Это позволяет держать итоговый образ небольшим и не включать в него инструменты сборки.

В **docker-compose.yml** аргумент передаётся так:

build:

context: .

dockerfile: Dockerfile

args:

SERVICE: booking

Архитектура контейнеризации

Проект состоит из следующих контейнеров:

Контейнер	Порт (внутренний)	Назначение
nginx	8080	API Gateway
auth-service	8081	Аутентификация
user-service	8082	Пользователи
property-service	8083	Объекты недвижимости
booking-service	8084	Бронирования
payment-service	8085	Платежи
review-service	8086	Отзывы

Контейнер	Порт (внутренний)	Назначение
<code>communication-service</code>	8087	Сообщения
PostgreSQL (×7)	—	Отдельная БД на каждый сервис
<code>kafka</code>	9092	Брокер сообщений
<code>zookeeper</code>	2181	Координатор Kafka
<code>kafka-ui</code>	9080	Веб-интерфейс Kafka

Сетевое взаимодействие

Межсервисные HTTP-запросы выполняются по внутренним адресам Docker Compose, например:

- `http://auth-service:8081`
- `http://user-service:8082`
- `http://property-service:8083`
- `http://booking-service:8084`
- `http://payment-service:8085`
- `http://review-service:8086`
- `http://communication-service:8087`

Такое решение позволяет сервисам общаться друг с другом внутри одной Docker-сети без использования внешних адресов. Адрес каждого соседнего сервиса передаётся через переменную окружения, например `AUTH_SERVICE_URL=http://auth-service:8081`.

Сервисы, использующие Kafka, зависят от контейнера `kafka` через `depends_on` с условием `service_healthy`, чтобы не стартовать раньше брокера.

API Gateway

Для входа в систему используется Nginx, который принимает запросы на порт `8080` и проксирует их к нужным сервисам под префиксом `/v1`.

Примеры маршрутов:

- `/v1/auth/` → `auth-service:8081`
- `/v1/users/` → `user-service:8082`
- `/v1/properties/` → `property-service:8083`

- `/v1/bookings/` → `booking-service:8084`
- `/v1/payments/` → `payment-service:8085`
- `/v1/reviews/` → `review-service:8086`
- `/v1/messages/` → `communication-service:8087`

Общие переменные окружения

В `docker-compose.yml` используется YAML-якорь для переиспользования общих переменных:

```
x-common-env: &common-env
```

```
SERVICE_TOKEN: "${SERVICE_TOKEN:-internal-service-token}"
```

```
KAFKA_BROKER: "${KAFKA_BROKER:-kafka:9092}"
```

Каждый сервис подключает этот блок через `<<: *common-env`, что исключает дублирование.

Итог по ЛР3

Требования лабораторной работы выполнены:

- реализован Dockerfile для всех сервисов на основе единого многоэтапного шаблона;
- подготовлен общий `docker-compose.yml`, поднимающий все сервисы, БД, Kafka и Gateway;
- настроено взаимодействие между сервисами через внутреннюю Docker-сеть по DNS-именам;
- добавлен Nginx как API Gateway для единой точки входа.

Что удобно указать в отчёте

1. Приложение состоит из набора микросервисов, каждый сервис запускается в отдельном контейнере.
2. Для хранения данных используется PostgreSQL, для обмена сообщениями — Kafka.
3. `docker-compose.yml` поднимает всю инфраструктуру целиком одной командой `docker compose up -d`.
4. Nginx выступает в роли API Gateway и объединяет сервисы под единым входом на порту `8080`.